

IXChange BV

SDR-SLC

SLC-DISC Service Discovery Specification

Field	Value
Document	SDR-SLC-DISC-1.0
Title	SLC-DISC Service Discovery Specification
Version	Protocol 1.0 Release 1.2 (function blocks, codec service type)
Status	RELEASE CANDIDATE
Date	September 2026
Author	Ivo van Ling, PA2IX
Organisation	IXChange BV
Licence	Creative Commons CC BY 4.0 — Open Source

Copyright © 2026 IXChange BV. All rights reserved. Published under Creative Commons CC BY 4.0.

Table of Contents

Revision History	3
1. Introduction	3
1.1 What changed in Release 1.2, and why.....	3
2. Normative Language	4
3. Terminology	4
4. Implementing SLC-DISC from Scratch	4
4.1 What SLC-DISC Actually Does.....	4
4.2 On STM32 with lwIP.....	5
4.3 On Linux (Python daemon).....	5
4.4 The Instance Name.....	6
4.5 When to Send the Announcement.....	6
5. Service Advertisement Parameters	7
5.1 Two service types.....	7
6. TXT Record Fields	7
6.1 Mandatory Fields.....	7
6.2 Optional Fields.....	8
6.3 Fields removed in Release 1.2.....	8
6.4 Complete TXT Record Example: transceiver.....	8
6.5 Complete TXT Record Example: codec service.....	9
6.6 One device on both types.....	9
7. Announcement Behaviour	9
7.1 On Link-Up.....	9
7.2 Responding to Queries.....	9
7.3 Clean Shutdown (Goodbye Packets).....	10
7.4 What is not advertised.....	10
8. IGMP and Multicast Configuration	10
8.1 Required Multicast Groups.....	10
8.2 STM32 MAC Filter Configuration.....	10
9. Platform Interoperability	10
9.1 macOS / Bonjour.....	10
9.2 Linux / Avahi.....	11
9.3 Windows.....	11
9.4 Known Interoperability Issues.....	11
10. Verification Procedures	11
10.1 Browse Test (macOS).....	11
10.2 Resolve Test (macOS).....	12
10.3 Address Resolution Test.....	12
10.4 TCP Connection Test.....	12
10.5 Packet Capture Verification.....	12
10.6 Avahi Browse Test (Linux).....	12
11. Conformance Requirements	13
11.1 Device.....	13
11.2 Client.....	13
12. Troubleshooting Guide	13
13. Security Considerations	14
13.1 mDNS Advertisement Spoofing.....	14
13.2 Future Authentication.....	14
14. References	14

Revision History

Rev	Date	Author	Description
1.0	2026-05-01	I. van Ling	Initial public draft.
1.1	2026-05-23	I. van Ling	Revised for implementers. Added §4 'Implementing SLC-DISC from Scratch' for STM32/IWIP and Raspberry Pi daemons. Detailed TXT record notes. Expanded IGMP section. Expanded verification section. Troubleshooting guide.
1.2	2026-09-14	I. van Ling	Aligned with SDR-SLC-CP-1.0 Release 1.3. TXT field <code>functions</code> replaces <code>role</code> , <code>rx</code> and <code>tx</code> . <code>fw</code> names the implementation. <code>stream</code> dropped. New <code>_slc-codec._tcp</code> service type with <code>codecs</code> hint. Client SHALL NOT reject unknown functions. Instance name is the device's durable identity. All examples, code snippets and verification outputs updated. Reference implementations named.
1.2	2026-09-16	I. van Ling	Re-issued alongside the SDR-SLC-CP-1.0 Release 1.3 and SDR-SLC-VITA-1.0 Release 1.2 amendments of the same date (connectors, <code>zero_point</code>). No content changes; cross-references to SDR-SLC-CP-1.0 verified against the amended section numbering.

WARNING	Release 1.2 changes the mandatory TXT field set and is not compatible with clients that require the Release 1.1 fields <code>role</code> , <code>rx</code> and <code>tx</code> . A device advertising the Release 1.1 fields is not conformant to this release. See §1.1.
----------------	---

1. Introduction

SLC-DISC is the service discovery layer of the SDR-SLC protocol suite. Its job is simple: allow an SDR-SLC client to find SDR-SLC devices on the local network automatically, without any manual IP address configuration, without a central server, and without any setup beyond plugging in the Ethernet cable.

SLC-DISC is built on two IETF standards that are natively supported on every platform that SDR-SLC targets. Multicast DNS (mDNS, RFC 6762) handles zero-configuration hostname resolution on the `.local.` domain. DNS-Based Service Discovery (DNS-SD, RFC 6763) adds structured service metadata that allows clients to learn device identity, which functions the device implements, its frequency range and its firmware before opening a control connection.

On macOS, this is Apple's Bonjour. On Linux, it is Avahi. On Windows 10 and later, it is built into the OS. None of these require any installation or configuration from the user. Plug in the hardware, and it appears in the client's device list.

This document specifies what the device must advertise, how it must behave on the network, and how a client finds and verifies it. It also provides a practical implementation guide for developers building their first SLC-DISC implementation. The two reference implementations, `sdr-slc-rtld` and `sdr-slc-bladerfd`, both advertise through Avahi and are the source of the expected outputs in §10.

1.1 What changed in Release 1.2, and why

SDR-SLC-CP-1.0 Release 1.2 introduced function blocks: a device implements an envelope plus any combination of `rx`, `tx` and `codec`. The Release 1.1 TXT fields `role=rx|tx|rxtx`, `rx=0|1`, `tx=0|1` could describe a radio and nothing else. A codec service — a device with no receiver and no transmitter — could not be advertised without lying, and widening the `role` value set would have made strict Release 1.1 clients drop the instance with an error.

So the field is replaced rather than extended: `functions=` carries a comma-separated list of function block identifiers, a client ignores identifiers it does not know, and a codec service lives on its own service type so that a client looking for radios never has to filter it out. The other changes are consequences: `fw` now names the implementation as well as the version, because two daemons at 1.0 were indistinguishable in a device list; and `stream` is dropped, because transports are negotiated in capabilities and the TXT value was never read.

2. Normative Language

Keyword	Meaning
SHALL	Absolute requirement. Non-conformant if omitted.
SHALL NOT	Absolute prohibition.
SHOULD	Strong recommendation.
MAY	Optional.

3. Terminology

Term	Definition
SLC-DISC	Service discovery layer. The complete mDNS/DNS-SD implementation on the device.
mDNS	Multicast DNS (RFC 6762). Resolves <code>.local.</code> hostnames without a central DNS server.
DNS-SD	DNS-Based Service Discovery (RFC 6763). Advertises and browses services by type.
PTR record	DNS Pointer record. Maps a service type to an instance name.
SRV record	DNS Service record. Maps an instance name to a hostname and port number.
TXT record	DNS Text record. Carries key=value metadata about a service instance.
A record	DNS Address record. Maps a hostname to an IPv4 address.
TTL	Time To Live. Seconds for which a DNS record may be cached by clients.
Instance name	The mDNS name identifying this specific device. Default format: <code>SLC-XXYYZZ</code> . The device's durable identity to clients.
Service type	The DNS-SD type a device advertises under. <code>_sdr-slc._tcp</code> for radios, <code>_slc-codec._tcp</code> for codec services.
Function block	A named set of capabilities defined in SDR-SLC-CP-1.0: <code>rx</code> , <code>tx</code> , <code>codec</code> , or a vendor-prefixed identifier.
Bonjour	Apple's mDNS/DNS-SD implementation. Native on macOS and iOS.
Avahi	Open-source mDNS/DNS-SD implementation for Linux.
IGMP	Internet Group Management Protocol. Used to join IPv4 multicast groups.
224.0.0.251	The mDNS IPv4 multicast group address.
5353	The mDNS UDP port.

4. Implementing SLC-DISC from Scratch

This chapter is for the developer implementing SLC-DISC for the first time on either an STM32 microcontroller or a Linux daemon. It explains exactly what you need to do, in what order, with concrete code guidance.

4.1 What SLC-DISC Actually Does

From the device's perspective, SLC-DISC does three things:

- On Ethernet link-up: join the mDNS multicast group 224.0.0.251 via IGMP, then send an unsolicited mDNS announcement advertising the device's presence.
- When a client sends a DNS query for `_sdr-slc._tcp.local.` (or `_slc-codec._tcp.local.`): respond with PTR, SRV, TXT, and A records describing the device.
- On clean shutdown: re-announce all records with TTL=0 (goodbye packets) to immediately remove the device from client caches.

That is all. No server, no registration, no configuration. The device is its own DNS server for its own name.

4.2 On STM32 with lwIP

The recommended approach on STM32 is to use the mDNS module included in lwIP 2.1+ (`mdns.c`). It is not compiled by default; you must enable it in `lwipopts.h`.

```
// In lwipopts.h:
#define LWIP_MDNS_RESPONDER      1
#define MDNS_MAX_SERVICES       2    // 1 for _sdr-slc_tcp, +1 if you also carry a codec
#define LWIP_IGMP                1    // required for mDNS multicast
#define LWIP_DNS                 1    // required by mdns.c

// In your network initialisation (after netif_add):
mdns_resp_init();
mdns_resp_add_netif(&gnetif, "SLC-000012", 120);    // hostname, TTL

// Register the _sdr-slc_tcp service:
mdns_resp_add_service(&gnetif,
    "SLC-000012",    // instance name
    "_sdr-slc",     // service type
    "_tcp",         // protocol
    4620,           // port
    slcdisc_txt_cb, // TXT record callback
    NULL);

// The TXT record callback for a receive-only device:
static void slcdisc_txt_cb(struct mdns_service *service, void *arg) {
    mdns_resp_add_service_txtitem(service, "proto=sdr-slc", 13);
    mdns_resp_add_service_txtitem(service, "proto_ver=1.0", 13);
    mdns_resp_add_service_txtitem(service, "name=SLC-000012", 15);
    mdns_resp_add_service_txtitem(service, "manufacturer=IXChange BV", 24);
    mdns_resp_add_service_txtitem(service, "hw=SLC-TAYLOE-1.0", 17);
    mdns_resp_add_service_txtitem(service, "fw=slc-tayloe/1.0.0", 19);
    mdns_resp_add_service_txtitem(service, "functions=rx", 12);
    mdns_resp_add_service_txtitem(service, "minhz=1800000", 13);
    mdns_resp_add_service_txtitem(service, "maxhz=30000000", 14);
}
```

TIP

The lwIP mDNS module handles the announcement, query responses, and goodbye packets automatically once you call `mdns_resp_add_netif()` and `mdns_resp_add_service()`. You do not need to implement UDP multicast send/receive yourself. The module also handles the required conflict detection (probing) before announcing. Count the bytes in each `txtitem` carefully; a length one short silently truncates the value.

NOTE

The hostname you pass to `mdns_resp_add_netif()` becomes the mDNS hostname (e.g. `SLC-000012.local.`). Generate it from the last 3 octets of your Ethernet MAC address in uppercase hex. This guarantees uniqueness on the LAN without any configuration.

4.3 On Linux (Python daemon)

For a Raspberry Pi or PC daemon bridging an RTL-SDR or other hardware, use the `zeroconf` Python library. It handles all the mDNS/DNS-SD complexity. (The C reference daemons use `avahi-client` directly; the record set is identical.)

```
from zeroconf import Zeroconf, ServiceInfo
import socket, uuid

def get_local_ip():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 80))
    ip = s.getsockname()[0]
    s.close()
    return ip

mac = uuid.getnode()
instance_name = "SLC-%06X" % (mac & 0xFFFFF)

zeroconf = Zeroconf()
info = ServiceInfo(
    "_sdr-slc_tcp.local.",
```

```
f"{instance_name}._sdr-slc._tcp.local.",
addresses=[socket.inet_aton(get_local_ip())],
port=4620,
properties={
    "proto": "sdr-slc",
    "proto_ver": "1.0",
    "name": instance_name,
    "manufacturer": "PA2IX",
    "hw": "RTL-SDR-V4",
    "fw": "sdr-slc-rtld/1.0",
    "functions": "rx",
    "minhz": "100000",
    "maxhz": "1766000000",
},
server=f"{instance_name}.local.",
)
zeroconf.register_service(info)
print(f"Announced {instance_name} on mDNS")

# On shutdown:
# zeroconf.unregister_service(info)      # sends goodbye packets
# zeroconf.close()
```

TIP

The zeroconf library handles everything: multicast group join, announcement, conflict detection, query responses, and goodbye packets. Install it with `pip3 install zeroconf`.

4.4 The Instance Name

The instance name is the unique identifier for your device on the mDNS network. The default format is SLC-XXYYZZ where XXYYZZ are the last three octets of the Ethernet MAC address in uppercase hexadecimal.

```
// STM32: derive instance name from MAC address
uint8_t mac[6];
HAL_ETH_GetMACAddress(&heth, mac);
char instance_name[16];
snprintf(instance_name, sizeof(instance_name),
    "SLC-%02X%02X%02X", mac[3], mac[4], mac[5]);
// Result: e.g. "SLC-12AB34"
// mDNS hostname: SLC-12AB34.local.
// mDNS instance name: SLC-12AB34._sdr-slc._tcp.local.
// TXT name field: name=SLC-12AB34
```

The instance name is the key under which a client stores a known device — its saved settings, its position in the device list, its calibration. It SHALL NOT change when the implementation is renamed or upgraded, and a device SHALL NOT let the mDNS responder rename it automatically on a collision: a collision on a MAC-derived name is a configuration error, and it SHOULD be logged rather than resolved by appending a digit and silently becoming a different device.

NOTE

Using the MAC address guarantees uniqueness without any configuration. If the user wants a human-friendly name, they can set it via a configuration interface, but the MAC-derived default must work out of the box.

4.5 When to Send the Announcement

The announcement sequence on link-up must be ordered correctly, especially on managed switches with IGMP snooping:

```
Link-up sequence (timing critical on managed switches):
t = 0 ms      Ethernet link established
t = 1 ms      IGMPv2 Membership Report sent to 224.0.0.251
t = 100 ms    Wait for switch IGMP table to update
t = 100 ms    mDNS announcement sent:
                PTR _sdr-slc._tcp.local. -> SLC-XXYYZZ._sdr-slc._tcp.local.
                SRV SLC-XXYYZZ._sdr-slc._tcp.local. :4620
                TXT SLC-XXYYZZ._sdr-slc._tcp.local. [key=value pairs]
                A SLC-XXYYZZ.local. -> [device IP]

CRITICAL: The IGMP report MUST be sent before the announcement.
```

WARNING

The 100 ms wait between the IGMP report and the mDNS announcement is not optional on managed switches. Many enterprise and prosumer switches need time to process the IGMP report and update their forwarding tables. Skipping the wait causes the announcement to be dropped, and the device will not be discovered until the client sends a query.

5. Service Advertisement Parameters

Parameter	Value
Service type, radio	<code>_sdr-slc._tcp.local.</code> — any device implementing rx or tx
Service type, codec	<code>_slc-codec._tcp.local.</code> — any device implementing codec
Transport	TCP
Default control port	4620
mDNS multicast group	224.0.0.251 (IPv4). IPv6 is not used.
mDNS port	5353/UDP
Record TTL	120 seconds (per RFC 6762 §11.3)
Instance name format	SLC-XXYYZZ (last 3 octets of MAC address, uppercase hex) by default
Example hostname	<code>SLC-000012.local.</code>
Example instance	<code>SLC-000012._sdr-slc._tcp.local.</code>

5.1 Two service types

A program looking for a radio browses `_sdr-slc._tcp` and sees only radios, with nothing to filter. A program that also wants a codec browses a second service type. That is one extra line of code, and it saves every client a filtering step that somebody will forget.

A device implementing rx or tx SHALL advertise on `_sdr-slc._tcp`. A device implementing codec SHALL advertise on `_slc-codec._tcp`. A device implementing both SHALL advertise two independent service instances, one on each type; they MAY share an instance name, hostname and port.

Clients SHALL NOT infer any relationship between a radio instance and a codec instance that share a name or an address. Sharing is informative: a client may display it ("radio and codec on one device"); it may not decide on it. Moving the codec to a different machine must require zero changes in client code.

6. TXT Record Fields

DNS-SD TXT records carry structured metadata about the service. The client reads these fields before opening a TCP control connection, allowing it to verify compatibility and display device identity to the operator. TXT records are hints sufficient to decide whether to connect; `get_capabilities` over SLC-CP is the truth.

6.1 Mandatory Fields

Every SDR-SLC service instance SHALL include all of these fields. A client that finds a service instance missing `proto` SHOULD not connect; a client that finds any other mandatory field missing SHOULD still display the instance, marked as incomplete, and MAY connect.

Field	Example value	Description
<code>proto</code>	<code>sdr-slc</code>	Protocol family identifier. Always the literal string <code>sdr-slc</code> . Allows clients to distinguish SDR-SLC services from other <code>_tcp</code> services.
<code>proto_ver</code>	<code>1.0</code>	Protocol version, as negotiated in SLC-CP <code>hello</code> . Only <code>1.0</code> is defined. Document release numbers (1.1, 1.2, 1.3) SHALL NOT appear here.
<code>name</code>	<code>SLC-000012</code>	Human-readable device name. Same as the mDNS instance name by default. MAY be overridden by user configuration.

Field	Example value	Description
manufacturer	IXChange BV	Hardware manufacturer name.
hw	SLC-TAYLOE-1.0	Hardware revision string. Manufacturer-defined.
fw	sdr-slc-rtld/1.0	<implementation>/<version>. The implementation name distinguishes two daemons at the same version in a device list. A client MAY display it verbatim; it SHALL NOT parse it for any decision.
functions	rx,tx	Comma-separated list of function block identifiers the device implements: any of rx, tx, codec, or vendor-prefixed identifiers. No spaces.

Clients SHALL ignore functions identifiers they do not recognise. Clients SHALL NOT reject a service instance because it advertises a function they do not know. That sentence is the whole forward compatibility of discovery: without it, every existing client trips over the first new kind of device.

6.2 Optional Fields

Field	Example value	Description
model	SDR-SLC HF Transceiver V1.0	Full hardware model identifier for display in the client device list.
minhz	1800000	Minimum receivable frequency in Hz. Decimal string. Clients SHALL parse it as a 64-bit unsigned integer. Radio instances only.
maxhz	30000000	Maximum receivable frequency in Hz. Same rules as minhz.
codecs	codec2,ambe	Codec instances only. A coarse hint of codec families available, sufficient to decide whether to connect. MAY be incomplete; the <code>formats[]</code> and <code>conversions[]</code> lists in <code>get_capabilities</code> are normative.

WARNING	minhz and maxhz are DNS TXT string values containing decimal integers. Clients SHALL parse them as 64-bit unsigned integers. A 32-bit integer overflows at approximately 4.295 GHz. Hardware supporting frequencies above 4.3 GHz MUST express these values as full decimal strings (e.g. "10000000000" for 10 GHz).
----------------	--

NOTE	TXT records have a size limit and cannot express which combinations of codec, mode and rate are valid. That is why the <code>codecs</code> hint is deliberately coarse and the authoritative list travels over the control plane. The same pattern applies to a radio: minhz and maxhz are enough to decide whether to connect; <code>frequency_ranges_hz</code> in capabilities is the truth.
-------------	--

6.3 Fields removed in Release 1.2

role, rx and tx are replaced by functions. stream is removed: the transports a function carries are advertised in capabilities. <block>. transports[], and the TXT value was never read by any client. A device SHALL NOT emit these fields. A client MAY accept them from a Release 1.1 device by mapping role=rx to functions=rx, role=tx to functions=tx and role=rxtx to functions=rx, tx, and SHOULD log that the responder predates Release 1.2.

6.4 Complete TXT Record Example: transceiver

The complete DNS record set for a transceiver with instance name SLC-000012 at IP 192.168.5.35 on port 4620:

```
; PTR: service browse answer
_sdr-slc._tcp.local. PTR SLC-000012._sdr-slc._tcp.local.

; DNS-SD meta-query: service type enumeration
_services._dns-sd._udp.local. PTR _sdr-slc._tcp.local.

; SRV: hostname and port
SLC-000012._sdr-slc._tcp.local. SRV 0 0 4620 SLC-000012.local.

; TXT: metadata
SLC-000012._sdr-slc._tcp.local. TXT
    proto=sdr-slc
```

```

proto_ver=1.0
name=SLC-000012
manufacturer=IXChange BV
hw=SLC-TAYLOE-1.0
fw=slc-tayloe/1.0.0
functions=rx,tx
model=SDR-SLC HF Transceiver V1.0
minhz=1800000
maxhz=30000000

; A: hostname resolution
SLC-000012.local. A 192.168.5.35

```

6.5 Complete TXT Record Example: codec service

A codec service on a Linux box, no radio:

```

_slc-codec._tcp.local. PTR NUC-Codec-01._slc-codec._tcp.local.
_services._dns-sd._udp.local. PTR _slc-codec._tcp.local.
NUC-Codec-01._slc-codec._tcp.local. SRV 0 0 4620 NUC-Codec-01.local.
NUC-Codec-01._slc-codec._tcp.local. TXT
    proto=sdr-slc
    proto_ver=1.0
    name=NUC-Codec-01
    manufacturer=PA2IX
    hw=NUC-i5
    fw=slc-codecd/0.3
    functions=codec
    codecs=codec2,ambe
NUC-Codec-01.local. A 192.168.5.40

```

6.6 One device on both types

A radio board that also carries a vocoder chip publishes two independent instances:

```

_sdr-slc._tcp name=SLC-UHF01 functions=rx,tx fw=sdr-slc-bladerfd/1.0 ...
_slc-codec._tcp name=SLC-UHF01 functions=codec codecs=ambe fw=sdr-slc-bladerfd/1.0 ...

```

Both point at the same hostname and port. A client that browses only `_sdr-slc._tcp` sees a radio and nothing else, which is correct.

7. Announcement Behaviour

7.1 On Link-Up

- Send IGMPv2 Membership Report for 224.0.0.251.
- Wait at least 100 ms.
- Send unsolicited mDNS announcement (all four record types: PTR, SRV, TXT, A) for every service instance the device advertises.

7.2 Responding to Queries

Query type	Records returned
PTR for <code>_sdr-slc._tcp.local.</code>	PTR + SRV + TXT + A (the full set) for each radio instance
PTR for <code>_slc-codec._tcp.local.</code>	PTR + SRV + TXT + A for each codec instance
PTR for <code>_services._dns-sd._udp.local.</code>	PTR for each service type the device advertises
SRV for instance name	SRV + A
TXT for instance name	TXT
A for hostname (SLC-XXYYZZ.local.)	A

Responses SHALL be sent to the mDNS multicast group (224.0.0.251:5353) with the QR bit set. Known-answer suppression per RFC 6762 §7 SHOULD be implemented to avoid redundant responses.

7.3 Clean Shutdown (Goodbye Packets)

On clean shutdown the device SHOULD re-announce all DNS records with TTL=0. Clients that receive a goodbye packet remove the service from their cache immediately, preventing the client from showing a stale entry for the TTL duration (120 seconds). If power is abruptly removed, clients will purge the entry when the TTL expires.

7.4 What is not advertised

Ownership and availability (control_state, busy) change on a timescale the mDNS cache cannot follow, and are reported over the control plane only (SDR-SLC-CP-1.0 §17.3). A stale "free" in a TXT record is worse than no information: it looks like something a client can act on. Likewise authentication requirements are learned from hello and get_capabilities, not from TXT.

8. IGMP and Multicast Configuration

8.1 Required Multicast Groups

Group	IPv4 address	MAC address	Purpose
mDNS	224.0.0.251	01:00:5E:00:00:FB	SLC-DISC announcement and query response.
PTPv2 (opt.)	224.0.1.129	01:1B:19:00:00:00	PTPv2 SYNC/FOLLOW_UP/ANNOUNCE (if implemented).

8.2 STM32 MAC Filter Configuration

The STM32H7 Ethernet MAC must be configured to receive multicast frames addressed to 01:00:5E:00:00:FB. This requires setting up the MAC multicast hash filter correctly.

```
// After calling HAL_ETH_Start() and lwIP igmp_joingroup():
// Simplest approach: pass all multicast (for development/testing)
ETH_MACFilterConfigTypeDef filter = {0};
filter.PassAllMulticast = ENABLE;
HAL_ETH_SetMACFilterConfig(&heth, &filter);

// For production: use the hash filter and set only the required bits.
// The mDNS MAC 01:00:5E:00:00:FB hashes to bit 32 in the hash table.
// Consult the STM32H7 reference manual for MACTHR/MACHT1R.

// Also required: join the IGMP group via lwIP
ip4_addr_t mcast_addr;
IP4_ADDR(&mcast_addr, 224, 0, 0, 251);
igmp_joingroup(netif_ip4_addr(&gnetif), &mcast_addr);
```

NOTE

The HAL_ETH_SetMACFilterConfig() call that lwIP makes internally may overwrite your custom hash filter settings. If you find multicast reception stopping after lwIP runs, call your filter configuration after the lwIP network interface initialisation is complete.

9. Platform Interoperability

9.1 macOS / Bonjour

macOS includes native Bonjour mDNS/DNS-SD support. The SDR-SLC client uses the Network framework (NWBrowser) to browse _sdr-slc._tcp.local. and _slc-codec._tcp.local. with one browser each. Bonjour caches DNS records for their TTL period. After a device goes offline without sending goodbye packets, macOS will attempt to

reconfirm the record at 80% and 90% of TTL before expiring it, so the entry persists for up to 120 seconds after the device disconnects.

A sandboxed macOS or iOS client requires the Local Network privacy entitlement and a `NSBonjourServices` list naming both service types in its `Info.plist`; without the second entry, codec services are invisible to it and no error is reported.

9.2 Linux / Avahi

Avahi is the reference implementation on Linux, and what both reference daemons publish through.

```
# Browse for SDR-SLC radios:
avahi-browse _sdr-slc._tcp
# Browse for codec services:
avahi-browse _slc-codec._tcp
# With resolution (-r) shows hostname and TXT:
avahi-browse -r _sdr-slc._tcp
# Resolve a specific instance:
avahi-resolve -n SLC-000012.local.
```

9.3 Windows

Windows 10 version 1703+ includes a built-in mDNS client. Third-party clients may use the Bonjour SDK for Windows for DNS-SD browsing.

9.4 Known Interoperability Issues

Issue	Platforms affected	Mitigation
IGMP snooping without querier blocks mDNS multicast.	Managed enterprise switches.	Enable IGMP querier on the VLAN, or disable IGMP snooping for the VLAN.
mDNS disabled by MDM policy.	Windows in corporate domains.	Use static IP fallback; document the configuration procedure.
Avahi daemon not running.	Minimal Linux installations.	<code>sudo apt install avahi-daemon</code>
IPv6 mDNS (ff02::fb) not forwarded.	Some managed switches.	Use IPv4 multicast only; SDR-SLC uses IPv4 exclusively.
Firewall blocking UDP 5353.	Any platform with local firewall.	Allow UDP 5353 on the LAN interface.
Client sandbox blocks Bonjour.	macOS / iOS App Store builds.	Local Network entitlement and <code>NSBonjourServices</code> with both service types.

10. Verification Procedures

Use these procedures to verify that your SLC-DISC implementation is working correctly. Run them from a macOS or Linux host on the same LAN as the device.

10.1 Browse Test (macOS)

```
dns-sd -B _sdr-slc._tcp local.
```

Expected output within a few seconds:

```
Browsing for _sdr-slc._tcp.local.
```

```
Timestamp    A/R Flags if Domain    Service Type    Instance Name
09:12:15.001 Add      2   14 local.    _sdr-slc._tcp    SLC-000012
```

If nothing appears within 10 seconds:

1. Check that the device Ethernet link is up.
2. Check that the device is sending IGMPv2 reports (Wireshark: igmp).
3. Check that the mDNS announcement is being sent (Wireshark: udp port 5353).
4. Check IGMP snooping configuration on the managed switch.

10.2 Resolve Test (macOS)

```
dns-sd -L "SLC-000012" _sdr-slc._tcp local.
```

Expected output:

```
Lookup SLC-000012._sdr-slc._tcp.local.
09:12:20.000 SLC-000012._sdr-slc._tcp.local. can be reached at
                SLC-000012.local.:4620 (interface 14)
  proto=sdr-slc proto_ver=1.0 name=SLC-000012
manufacturer=IXChange BV hw=SLC-TAYLOE-1.0 fw=slc-tayloe/1.0.0
functions=rx,tx model=SDR-SLC HF Transceiver V1.0
minhz=1800000 maxhz=30000000
```

Verify: port is 4620, all mandatory TXT fields (section 6.1) are present, functions lists exactly the blocks that get_capabilities will return.

10.3 Address Resolution Test

```
dns-sd -G v4 SLC-000012.local.
09:12:25.000 SLC-000012.local. Add ... 192.168.5.35
```

```
ping -c 3 SLC-000012.local.
Expected: 3 successful ping replies from 192.168.5.35
```

10.4 TCP Connection Test

```
# Verify SLC-CP is listening on port 4620:
nc -z SLC-000012.local. 4620 && echo "TCP port 4620 open"

# Minimal hello exchange (length-prefixed framing, SLC-CP section 8.3):
python3 - << 'EOF'
import socket, struct, json
m = json.dumps({"type":"request","id":"t1","command":"hello","params":
    {"client_name":"test","client_version":"0","supported_protocol_versions":["1.0"]}}).encode()
s = socket.create_connection(("SLC-000012.local.", 4620))
s.sendall(struct.pack(">I", len(m)) + m)
n, = struct.unpack(">I", s.recv(4)); print(s.recv(n).decode())
EOF

# Expected: a JSON response with "selected_protocol_version":"1.0".
# Cross-check: the functions listed in TXT must match the keys of
# get_capabilities (rx, tx, codec). A mismatch is a device bug.
```

10.5 Packet Capture Verification

```
# Capture all mDNS traffic from the device:
tcpdump -i eth0 -n 'src 192.168.5.35 and udp port 5353'
# Capture IGMP from the device:
tcpdump -i eth0 -n 'igmp and src 192.168.5.35'
# You should see: 224.0.0.251 > 224.0.0.251: igmp v2 report
# Verify PTR record in mDNS response:
tcpdump -i eth0 -n 'udp port 5353' -A | grep -e sdr-slc -e slc-codec
```

10.6 Avahi Browse Test (Linux)

```
avahi-browse -r _sdr-slc._tcp
```

Expected output for sdr-slc-rtld:

```
+ eth0 IPv4 SLC-5A5353 _sdr-slc._tcp local
= eth0 IPv4 SLC-5A5353 _sdr-slc._tcp local
  hostname = [SLC-5A5353.local]
  address = [192.168.1.181]
  port = [4620]
  txt = ["maxhz=1766000000" "minhz=100000" "functions=rx"
        "fw=sdr-slc-rtld/1.0" "hw=RTL-SDR-V4" "manufacturer=PA2IX"]
```

```
"name=SLC-5A5353" "proto_ver=1.0" "proto=sdr-slc"]
```

11. Conformance Requirements

11.1 Device

- Advertise on `_sdr-slc._tcp.local.` if implementing rx or tx; on `_slc-codec._tcp.local.` if implementing codec; on both, as separate instances, if implementing both. (SHALL)
- Use an instance name derived from the MAC address (SLC-XXYYZZ) by default; never auto-rename on collision. (SHALL)
- Include all mandatory TXT record fields (§6.1), with functions listing exactly the blocks `get_capabilities` returns. (SHALL)
- Emit fw as `<implementation>/<version>`. (SHALL)
- Not emit role, rx, tx or stream. (SHALL)
- Send IGMPv2 Membership Report for 224.0.0.251 before the mDNS announcement. (SHALL)
- Wait at least 100 ms between IGMP report and mDNS announcement. (SHOULD)
- Respond to PTR, SRV, TXT, and A queries. (SHALL)
- Set record TTL to 120 seconds. (SHALL)
- Send goodbye packets (TTL=0) on clean shutdown. (SHOULD)
- Implement RFC 6762 known-answer suppression. (SHOULD)

11.2 Client

- Browse `_sdr-slc._tcp.local.` before connecting to any radio; browse `_slc-codec._tcp.local.` if codec services are wanted. (SHALL)
- Verify `proto=sdr-slc` in the TXT record before connecting. (SHOULD)
- Ignore functions identifiers it does not recognise; never reject an instance for advertising one. (SHALL)
- Display an instance with missing optional or non-`proto` mandatory fields rather than dropping it. (SHOULD)
- Parse `minhz` and `maxhz` as 64-bit unsigned integers. (SHALL)
- Treat the instance name as the device's durable identity; store settings under it. (SHOULD)
- Handle service loss gracefully (TTL expiry or goodbye packet). (SHALL)
- Retry connection with exponential backoff after connection failure. (SHOULD)
- Display device name, manufacturer, functions and fw from the TXT record. (SHOULD)
- Not infer a relationship between a radio instance and a codec instance from a shared name or address. (SHALL)

12. Troubleshooting Guide

Symptom	Most likely cause	How to check and fix
Device not discovered at all	IGMP report not sent, or mDNS announcement not sent.	Wireshark on switch port: look for IGMP and UDP 5353 packets from the device IP within 200 ms of link-up.
Device discovered but immediately disappears	Goodbye packets sent unintentionally, or TTL set to 0.	Wireshark: filter <code>udp port 5353 and src device_ip</code> ; look for TTL=0 in the additional records.
Device shown as "Unknown" or with no functions	TXT callback emits Release 1.1 role/rx/tx, or functions missing.	<code>dns-sd -L</code> and compare against §6.1.
Codec service not shown	Client browses only <code>_sdr-slc._tcp</code> ; or a sandboxed client lacks <code>_slc-codec._tcp</code> in <code>NSBonjourServices</code> .	<code>avahi-browse _slc-codec._tcp</code> from another host to confirm the device side; then check the client's entitlement.
Two devices look identical in the	fw carries only a version	Emit fw= <code><implementation>/<version></code> .

Symptom	Most likely cause	How to check and fix
list	number.	
Device found on macOS but not Linux	IPv6 mDNS issue while IPv4 works.	Try <code>avahi - browse -a</code> to see all address families.
Device found on unmanaged switch but not managed	IGMP snooping blocking multicast without querier.	Enable IGMP querier on the switch VLAN, or temporarily disable snooping to confirm.
Port 4620 shown in resolution but TCP refused	SLC-CP server not started, or bound to the wrong interface.	<code>nc -z SLC-000012.local. 4620</code> . On STM32: confirm the lwIP TCP PCB is listening after <code>netif_add</code> .
minhz/maxhz shown incorrectly in client	Parsed as 32-bit integer, truncated.	Verify the client uses <code>uint64_t</code> / <code>BigInt</code> for these fields.
Device renamed itself SLC - 000012 (2)	mDNS responder auto-resolved a name collision.	Two devices share the last three MAC octets, or a stale entry is cached. Fix the name; do not accept the rename.

13. Security Considerations

13.1 mDNS Advertisement Spoofing

mDNS advertisements are unauthenticated. Any host on the LAN can advertise a rogue `_sdr-slc._tcp.local` service and redirect clients to an attacker-controlled host. Clients SHOULD verify the `proto` and `manufacturer` TXT record fields before connecting and SHOULD display device identity to the operator for confirmation.

Authentication of the device to the client is not provided at this layer. Authentication of the client to the device, for transmit functions, is provided by SDR-SLC-CP-1.0 §26; a rogue advertisement cannot obtain the pre-shared key from a client, because the client never sends it — it sends an HMAC over a nonce the device chose.

13.2 Future Authentication

A future version of SLC-DISC MAY add device authentication at the TXT record level, for example a signed device certificate fingerprint that the client can verify. This is not required for Protocol 1.0, which targets trusted LAN operation.

14. References

Reference	Title	Relevance
RFC 6762	Multicast DNS	mDNS protocol specification. Definitive reference.
RFC 6763	DNS-Based Service Discovery	DNS-SD protocol specification.
RFC 2782	DNS SRV records	SRV record format.
RFC 2119	Key Words for Use in RFCs	Normative keyword definitions.
SDR-SLC-CP-1.0 Release 1.3	SLC-CP Control Plane	Function blocks, <code>get_capabilities</code> , authentication; the TCP service advertised by SLC-DISC.
SDR-SLC-VITA-1.0 Release 1.2	VITA-SLC Stream Transport	I/Q stream transport.
SDR-SLC-RTP-1.0	SLC-RTP Framed Media Transport	Codec service transport.
SDR-SLC-PS-1.0 Release 1.2	Protocol Suite Architecture	Overall architecture and design rationale.
<code>sdr-slc-rtld</code> , <code>sdr-slc-bladerfd</code>	Reference implementations	Avahi-based advertisement; source of the expected outputs in §10.
<code>zeroconf</code> (Python)	<code>zeroconf</code> Python library	Recommended for Python daemon implementations.
<code>lwip mdns.c</code>	lwIP mDNS module	Recommended for STM32/embedded implementations.